

EO123030 G THARANYA

```

import os
import zipfile
from tensorflow.keras.applications import VGG16
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# 1. Download Dataset (Requires Kaggle API configuration)
# !kaggle datasets download -d paultimothymooney/chest-xray-pneumonia
!kaggle datasets download -d paultimothymooney/chest-xray-pneumonia

# 2. Extract Dataset
with zipfile.ZipFile("chest-xray-pneumonia.zip", "r") as zip_ref:
    zip_ref.extractall("chest_xray_dataset")

# Define paths
train_dir = "chest_xray_dataset/chest_xray/train"
val_dir = "chest_xray_dataset/chest_xray/val"
test_dir = "chest_xray_dataset/chest_xray/test"

```

Dataset URL: <https://www.kaggle.com/datasets/paultimothymooney/chest-xray-pneumonia>
 License(s): other
 Downloading chest-xray-pneumonia.zip to /content
 100% 2.29G/2.29G [01:00<00:00, 40.8MB/s]

```

# Create data generators
train_datagen = ImageDataGenerator(
    rescale=1.0 / 255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True
)

val_datagen = ImageDataGenerator(rescale=1.0 / 255)

# Generate batches
train_generator = train_datagen.flow_from_directory(
    train_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary'
)

val_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary'
)

```

Found 5216 images belonging to 2 classes.
 Found 16 images belonging to 2 classes.

```

# Load pre-trained VGG16
base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze convolutional layers
for layer in base_model.layers:
    layer.trainable = False

# Add custom heads
x = Flatten()(base_model.output)
x = Dense(256, activation='relu')(x)
output = Dense(1, activation='sigmoid')(x)

model = Model(inputs=base_model.input, outputs=output)

# Compile and Train
model.compile(optimizer=Adam(learning_rate=0.001), loss='binary_crossentropy', metrics=['accuracy'])
model.fit(train_generator, validation_data=val_generator, epochs=3)

```

```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16\_weights\_tf\_dim\_ordering\_and\_dilation.keras.h5
58889256/58889256 ————— 2s 0us/step
Epoch 1/3
163/163 ————— 143s 782ms/step - accuracy: 0.8978 - loss: 0.2883 - val_accuracy: 0.8125 - val_loss: 0.3698
Epoch 2/3
163/163 ————— 121s 737ms/step - accuracy: 0.9340 - loss: 0.1727 - val_accuracy: 0.7500 - val_loss: 0.3698
Epoch 3/3
163/163 ————— 122s 744ms/step - accuracy: 0.9419 - loss: 0.1531 - val_accuracy: 0.7500 - val_loss: 0.3698
<keras.src.callbacks.history.History at 0x7c441ab0c6b0>

```

```

# Load pre-trained VGG16
base_model_ft = VGG16(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Unfreeze all layers for fine-tuning
for layer in base_model_ft.layers:
    layer.trainable = True

x = Flatten()(base_model_ft.output)
x = Dense(256, activation='relu')(x)
output = Dense(1, activation='sigmoid')(x)

model_ft = Model(inputs=base_model_ft.input, outputs=output)

# Compile with lower learning rate (1e-5)
model_ft.compile(optimizer=Adam(learning_rate=1e-5), loss='binary_crossentropy', metrics=['accuracy'])
model_ft.fit(train_generator, validation_data=val_generator, epochs=3)

```

```

Epoch 1/3
163/163 ————— 199s 905ms/step - accuracy: 0.8921 - loss: 0.2352 - val_accuracy: 0.7500 - val_loss: 0.3698
Epoch 2/3
163/163 ————— 145s 888ms/step - accuracy: 0.9494 - loss: 0.1311 - val_accuracy: 0.7500 - val_loss: 0.3698
Epoch 3/3
163/163 ————— 147s 899ms/step - accuracy: 0.9618 - loss: 0.0964 - val_accuracy: 0.7500 - val_loss: 0.3698
<keras.src.callbacks.history.History at 0x7c43ec119280>

```

```

test_datagen = ImageDataGenerator(rescale=1.0 / 255)
test_generator = test_datagen.flow_from_directory(
    test_dir,
    target_size=(224, 224),
    batch_size=32,
    class_mode='binary'
)

test_loss, test_accuracy = model.evaluate(test_generator)
print(f"Test Loss: {test_loss:.4f}, Test Accuracy: {test_accuracy:.4f}")

```

```

Found 624 images belonging to 2 classes.
20/20 ————— 7s 306ms/step - accuracy: 0.8958 - loss: 0.3698
Test Loss: 0.3698, Test Accuracy: 0.8958

```